

Module 2: Mobile Data Management

Lecture1.Contents provider: In [Android](#), Content Providers are a very important [component](#) that serves the purpose of a relational database to store the data of applications. The role of the content provider in the android system is like a central repository in which data of the applications are stored, and it facilitates other applications to securely access and modifies that data based on the user requirements.

1.Uri: **Content URI(Uniform Resource Identifier)** is the key concept of Content providers. To access the data from a content provider, URI is used as a query string.

Structure of a Content URI: *content://authority/optionalPath/optionalID*

Details of different parts of Content URI:

- **content://** – It's always present, and is the scheme portion of the URI.
- **authority** – It is the unique name of the content provider, like photos, contacts. It's a string that can identify the whole content provider.
- **path** – It is often used to identify some or the other data of the provider. The path is mostly used to identify individual tables.
- **optional_id** – id is used to access a single particular record of a file. We use this only in cases where we need to access only a particular record and not the complete file. It's a numeric identifier to access a particular row of the data table.

Lecture2.CRUD access:

CRUD refers to the four basic operations a software application should be able to perform – Create, Read, Update, and Delete.

In such apps, users must be able to **create data**, have access to the data in the UI by **reading** the data, **update** or **edit** the data, and **delete** the data.

In full-fledged applications, CRUD apps consist of 3 parts: an API (or server), a database, and a user interface (UI).

The API contains the code and methods, the database stores and helps the user retrieve the information, while the user interface helps users interact with the app.

Browser: Android provides an awesome UI widget known as WebView. It is used to load web pages and view HTML data. We will use WebView class to make a simple browser with some functionality.

How to Make Simple Browser in Android

Some methods of WebView class that perform following functions.

loadUrl(): It loads a web page from url

canGoBack(): It checks if previous page history is available

goBack(): Go to previous page

canGoForward(): It checks if next page history is available

goForward(): Go to next page

reload(): Reload current page

stopLoading(): stop loading of current page

This browser will allow you to do following things.

- Open a web page url.
- Cancel loading of a web page.
- Go to previous or next web page.
- Reload a web page.
- See the progress of loading of web page in progress bar.
- Activity.main.xml

```
• <?xml version="1.0" encoding="utf-8"?>
  <LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity" >
    <WebView
      android:id="@+id/WebView1"
      android:layout_width="match_parent"
      android:layout_height="match_parent" />
  </LinearLayout>
```

-
- AndriodManifest.xml

```
• <?xml version="1.0" encoding="utf-8"?>
  <manifest xmlns:android="http://schemas.android.com/apk/res/android"

    package="com.example.myapplication">
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
      android:allowBackup="true"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:roundIcon="@mipmap/ic_launcher_round"
      android:supportsRtl="true"
      android:theme="@style/Theme.MyApplication">
      <activity

        android:name=".MainActivity"
        android:exported="true">
```

```

        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category
                android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

</application>

</manifest>

```

MainActivity.java

- ```

package com.example.myapplication;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.webkit.WebView;

public class MainActivity extends AppCompatActivity {
 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main);
 WebView w = findViewById(R.id.WebView1);
 w.loadUrl("https://www.google.com");
 }

```

**Lecture3.CallLog:** To get call history programmatically first add read contact permission in Manifest file:

```
<uses-permission android:name="android.permission.READ_CONTACTS" />
```

**Create xml file. Add the below code in xml file:**

```

<LinearLayout android:layout_height="fill_parent"
 android:layout_width="fill_parent"
 android:orientation="vertical">
 <TextView android:id="@+id/call"
 android:layout_height="fill_parent"
 android:layout_width="fill_parent">
 </TextView>
</LinearLayout>

```

**Now call the getCallDetails() method in java class:**

```

private void getCallDetails() {
 StringBuffer sb = new StringBuffer();
 Cursor managedCursor = managedQuery(CallLog.Calls.CONTENT_URI,null, null,null,
 null);
 int number = managedCursor.getColumnIndex(CallLog.Calls.NUMBER);
 int type = managedCursor.getColumnIndex(CallLog.Calls.TYPE);
 int date = managedCursor.getColumnIndex(CallLog.Calls.DATE);
 int duration = managedCursor.getColumnIndex(CallLog.Calls.DURATION);
 sb.append("Call Details :");
 while (managedCursor.moveToNext()) {
 String phNumber = managedCursor.getString(number);
 String callType = managedCursor.getString(type);
 String callDate = managedCursor.getString(date);
 Date callDayTime = new Date(Long.valueOf(callDate));
 String callDuration = managedCursor.getString(duration);
 String dir = null;
 int dircode = Integer.parseInt(callType);
 switch(dircode) {
 case CallLog.Calls.OUTGOING_TYPE:
 dir = "OUTGOING";
 break;

 case CallLog.Calls.INCOMING_TYPE:
 dir = "INCOMING";
 break;

 case CallLog.Calls.MISSED_TYPE:
 dir = "MISSED";
 break;
 }
 sb.append("\nPhone Number:--- "+phNumber+" \nCall Type:--- "+dir+" \nCall Date:---
"+callDayTime+" \nCall duration in sec :--- "+callDuration);
 sb.append("\n-----");
 }
 managedCursor.close();
 call.setText(sb);
}

```

#### **Lecture4. Contacts:**

```

<uses-permission android:name="android.permission.READ_CONTACTS"></uses-
permission>

```

#### *XML CODE*

```

<?xml version="1.0" encoding="utf-8"?>

```

<RelativeLayout

xmlns:android="<http://schemas.android.com/apk/res/android>"

xmlns:tools="<http://schemas.android.com/tools>"

android:id="@+id/relative\_layout"

android:layout\_width="match\_parent"

android:layout\_height="match\_parent"

tools:context=".MainActivity">

<!--Button to perform clicking event to

access contacts list inside listView-->

<Button

android:id="@+id/Button"

android:layout\_width="150dp"

android:layout\_height="52dp"

android:layout\_centerHorizontal="true"

android:layout\_margin="12dp"

android:background="#0F9D58"

android:text="Click Here"

android:textColor="#FFFFFF" />

<!--ListView to show all saved contacts-->

<ListView

android:id="@+id/ListView"

android:layout\_width="match\_parent"

android:layout\_height="match\_parent"

android:layout\_below="@+id/Button" />

</RelativeLayout>

### **JAVACODE:**

```
import android.database.Cursor;
import android.os.Bundle;
import android.provider.ContactsContract;
import android.view.View;
import android.widget.Button;
import android.widget.ListView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.cursoradapter.widget.SimpleCursorAdapter;
```

**Cursor:** To navigate through query results.

**ContactsContract:** Access to contact data on the device.

**ListView, Button, etc.:** UI components.

**SimpleCursorAdapter:** Binds data from a Cursor to a ListView

```
public class MainActivity extends AppCompatActivity {
 Cursor cursor;
 ListView listView;
 Button button;
```

**cursor:** To hold contact query results.

**listView:** To display contacts.

**button:** To trigger contact loading.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main); // Initializes the activity and loads the UI from
activity_main.xml.

 // declaring listView using findViewById()
 listView = findViewById(R.id.ListView);

 // declaring button using findViewById()
 button = findViewById(R.id.Button);
```

**//Finds the ListView and Button defined in the layout by their ID**

```
// set OnClickListener() to the button
button.setOnClickListener(new View.OnClickListener() {
 @Override
 public void onClick(View v) {
 // calling of getContacts()
 getContacts(); // When the button is clicked, the getContacts () method is called to load and display contacts.
 }
});
}
```

```
public void getContacts() {
```

```
 // create cursor and query the data
```

```
 cursor =
getContentResolver().query(ContactsContract.CommonDataKinds.Phone.CONTENT_URI,
null, null, null, null);
```

**// Queries the device's contact list (only phone contacts).**

**Returns a Cursor with all the contact data.**

```
startManagingCursor(cursor); // Automatically manages the cursor lifecycle
```

```
// data is a array of String type which is
// used to store Number ,Names and id.
```

```
String[] data = {ContactsContract.CommonDataKinds.Phone.NUMBER,
ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME,
ContactsContract.CommonDataKinds.Phone._ID};
int[] to = {android.R.id.text1, android.R.id.text2};
```

**// data: Which contact fields to fetch (phone number, name, ID).**

**to: IDs of views inside the layout android.R.layout.simple\_list\_item\_2 where data will be placed.**

```
// creation of adapter using SimpleCursorAdapter class
```

```
SimpleCursorAdapter adapter = new SimpleCursorAdapter(this,
android.R.layout.simple_list_item_2, cursor, data, to);
```

**//Binds the cursor data to a built-in layout with two text views.**

```

 // Calling setAdaptor() method to set created adapter
 listView.setAdapter(adapter);
 listView.setChoiceMode(ListView.CHOICE_MODE_MULTIPLE);
 }
}

```

**Displays the contacts in the `ListView`.**

**Enables multiple selection of contacts (if needed).**

## Lecture 5.Data Access and Storage: Shared Preferences:

**Shared Preferences** is the way in which one can store and retrieve small amounts of primitive data as key/value pairs to a file on the device storage such as String, int, float, Boolean that make up your preferences in an XML file inside the app on the device storage.

**Storage External:** Like internal storage, we are able to save or read data from the device external memory such as sdcard. The `FileInputStream` and `FileOutputStream` classes are used to read and write data into the file.

**Network Connection:**

To get the internet connection status, our app must acquire the **INTERNET** and **ACCESS\_NETWORK\_STATE** permissions. For that, we need to add the following permissions in the android manifest file like as shown below.

```

<manifest>

 <uses-permission android:name="android.permission.INTERNET"/>
 <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

</manifest>

```

In android, we can determine the internet connection status easily by using **getActiveNetworkInfo()** method of **ConnectivityManager** object.

```

ConnectivityManager cm =
 (ConnectivityManager)getApplicationContext().getSystemService(Context.CONNECTIVITY_SERVICE);
NetworkInfo nInfo = cm.getActiveNetworkInfo();
boolean connected = nInfo != null && nInfo.isAvailable() && nInfo.isConnected();

```

In android, we can easily determine the type of internet connection currently available i.e. either WI-FI or mobile data by using the **getType()** method of **NetworkInfo** object.

```
ConnectivityManager cm =
(ConnectivityManager)getApplicationContext().getSystemService(Context.CONNECTIVITY_SERVICE);
NetworkInfo nInfo = cm.getActiveNetworkInfo();
boolean isWiFi = nInfo.getType() == ConnectivityManager.TYPE_WIFI;
```

**Lecture 6-7.SQLite** - SQLite Databases: **SQLite** is an **open-source relational database** i.e. used to perform database operations on android devices such as storing, manipulating or retrieving persistent data from the database.

It is embedded in android by default. So, there is no need to perform any database setup or administration task. **SQLiteOpenHelper** class provides the functionality to use the SQLite database.

### SQLiteOpenHelper class

The `android.database.sqlite.SQLiteOpenHelper` class is used for database creation and version management. For performing any database operation, you have to provide the implementation of **onCreate()** and **onUpgrade()** methods of **SQLiteOpenHelper** class.

### *Methods of SQLiteOpenHelper class*

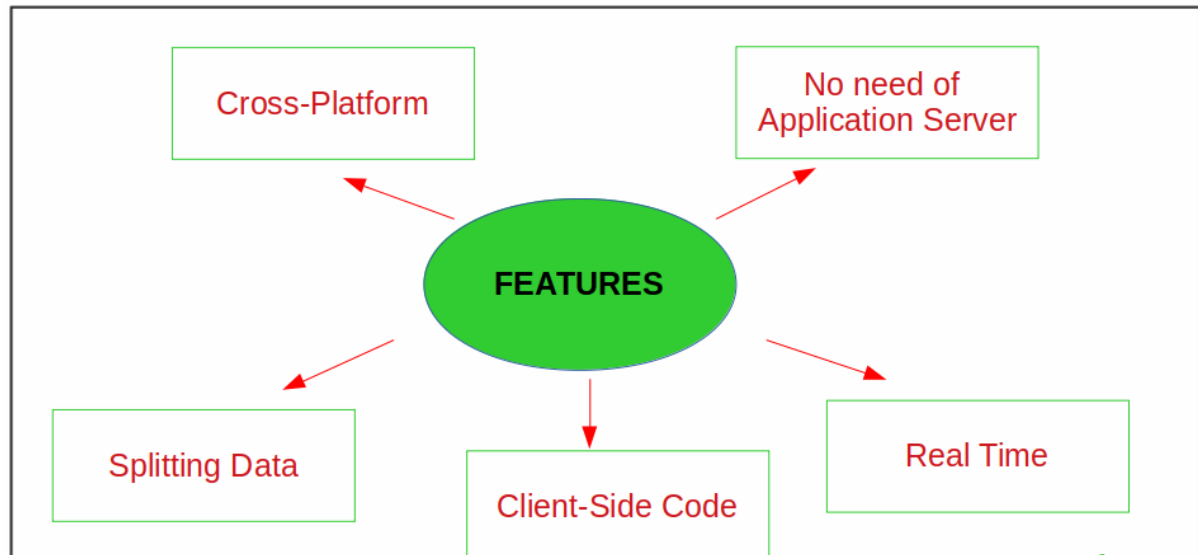
There are many methods in **SQLiteOpenHelper** class. Some of them are as follows:

Method	Description
<b>public abstract void onCreate(SQLiteDatabase db)</b>	called only once when database is created for the first time.
<b>public abstract void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)</b>	called when database needs to be upgraded.
<b>public synchronized void close ()</b>	closes the database object.
<b>public void onDowngrade(SQLiteDatabase db, int oldVersion, int newVersion)</b>	called when database needs to be downgraded.

### **Lecture 8-9. Introduction to Firebase with simple CRUD Operation:**

[Firebase](#) is a famous product of Google which is used by so many developers to add backend functionality for their website as well as apps. The Firebase will make your job really easier for the backend database and handling the database. [Firebase](#) is famous set of

backend that is used for **Serverless** or **Headless** web applications or mobile applications. In today's topic, we will discuss how to perform complete **CRUD** operation in firebase. CRUD operation stands for Create, Read, Update and Delete and these are the major building blocks of any web app or mobile app.



### Step by Step Implementation:

1. [How to Create and Add Data to Firebase Firestore in Android? - Geeks](https://www.geeksforgeeks.org/create-and-add-data-to-firebase-firestore-in-android/)  
[https://www.geeksforgeeks.org/create-and-add-data-to-firebase-firestore-in-android/forGeeks](https://www.geeksforgeeks.org/create-and-add-data-to-firebase-firestore-in-android/)

2. <https://www.geeksforgeeks.org/user-authentication-and-crud-operation-with-firebase-realtime-database-in-android/>

### Step by Step Implementation

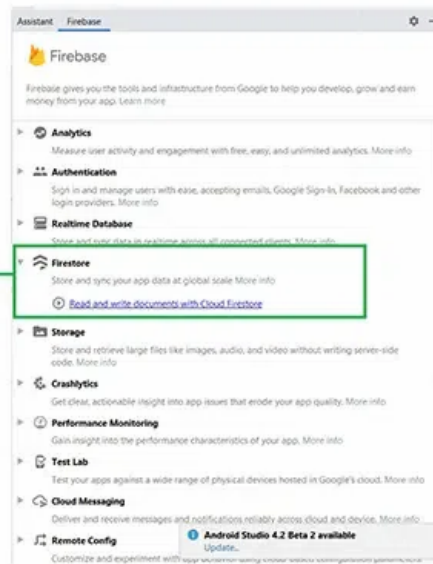
#### Step 1: Create a new Project

Create a new project in Android Studio and select **Java** as the programming language.

#### Step 2: Connect your app to Firebase

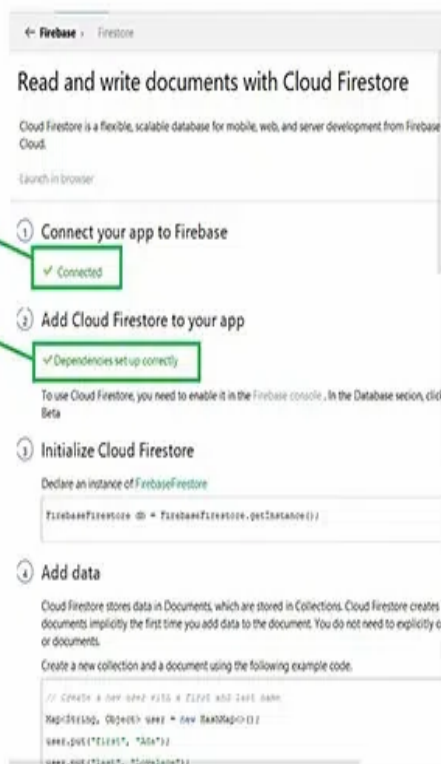
After creating a new project. Navigate to the Tools option on the top bar. Inside that click on Firebase. After clicking on Firebase, you can get to see the right column mentioned below in the screenshot.

After clicking on Tools>Firebase. You will get to see this window in right side. Inside this window click on Firestore option inside that click on Read and write documents with Cloud Firestore. Click on that option



Inside that column Navigate to Firebase Cloud Firestore. Click on that option and you will get to see two options on Connect app to Firebase and Add Cloud Firestore to your app. Click on Connect now option and your app will be connected to Firebase. After that click on the second option and now your App is connected to Firebase. After connecting your app to Firebase you will get to see the below screen.

After connection and adding Firebase Firestore to your app. You will get to see both the options as checked.



After that verify that dependency for Firebase Firestore database has been added to our Gradle file. Navigate to the **app > Gradle Scripts** inside that file. Check whether the below dependency is added or not. If the below dependency is not present in your build.gradle file. Add the below dependency in the dependencies section.

```
implementation 'com.google.firebase:firebase-firestore:22.0.1'
```

After adding this dependency sync your project and now we are ready for creating our app

### Step 3: Working with the AndroidManifest.xml file

For adding data to Firebase we should have to give permissions for accessing the internet. For adding these permissions navigate to the **app > AndroidManifest.xml**. Inside that file add the below permissions to it.

```
<!--Permissions for internet-->
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

### Step 4: Working with the activity\_main.xml file

Go to the **activity\_main.xml** file and refer to the following code. Below is the code for the **activity\_main.xml** file.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:orientation="vertical"
 tools:context=".MainActivity">

 <!--EditText for getting course Name-->
 <EditText
 android:id="@+id/idEdtCourseName"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:layout_marginStart="10dp"
 android:layout_marginTop="20dp"
 android:layout_marginEnd="10dp"
 android:hint="Course Name"
 android:importantForAutofill="no"
 android:inputType="text" />

 <!--EditText for getting course Duration-->
 <EditText
 android:id="@+id/idEdtCourseDuration"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:layout_marginStart="10dp"
 android:layout_marginTop="20dp"
 android:layout_marginEnd="10dp"
 android:hint="Course Duration in min"
 android:importantForAutofill="no"
```

```

 android:inputType="time" />

<!--EditText for getting course Description-->
<EditText
 android:id="@+id/idEdtCourseDescription"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:layout_marginStart="10dp"
 android:layout_marginTop="20dp"
 android:layout_marginEnd="10dp"
 android:hint="Course Description"
 android:importantForAutofill="no"
 android:inputType="text" />

<!--Button for adding your course to Firebase-->
<Button
 android:id="@+id/idBtnSubmitCourse"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:layout_margin="10dp"
 android:text="Submit Course Details"
 android:textAllCaps="false" />

</LinearLayout>

```

## Step 5: Create a new Java class for storing the data

For sending data to the Firebase Firestore database we have to create an Object class and send that whole object class to Firebase. For creating an object class navigate to the **app > java > your app's package name > Right-click on it and click on New > Java Class > Give a name to your class**. Here we named it **Courses** and add the below code to it.

Java

```

public class Courses {

 // variables for storing our data.
 private String courseName, courseDescription, courseDuration;

 public Courses() {
 // empty constructor
 // required for Firebase.
 }

 // Constructor for all variables.
 public Courses(String courseName, String courseDescription, String
courseDuration) {
 this.courseName = courseName;
 this.courseDescription = courseDescription;
 this.courseDuration = courseDuration;
 }

 // getter methods for all variables.

```

```

public String getCourseName() {
 return courseName;
}

public void setCourseName(String courseName) {
 this.courseName = courseName;
}

public String getCourseDescription() {
 return courseDescription;
}

// setter method for all variables.
public void setCourseDescription(String courseDescription) {
 this.courseDescription = courseDescription;
}

public String getCourseDuration() {
 return courseDuration;
}

public void setCourseDuration(String courseDuration) {
 this.courseDuration = courseDuration;
}
}

```

## Step 6: Working with the MainActivity.java file

Go to the **MainActivity.java** file and refer to the following code. Below is the code for the **MainActivity.java** file. Comments are added inside the code to understand the code in more detail.

Java

```

import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.firestore.CollectionReference;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.FirebaseFirestore;

public class MainActivity extends AppCompatActivity {

 // creating variables for our edit text
 private EditText courseNameEdt, courseDurationEdt, courseDescriptionEdt;

```

```

// creating variable for button
private Button submitCourseBtn;

// creating a strings for storing
// our values from edittext fields.
private String courseName, courseDuration, courseDescription;

// creating a variable
// for firebasefirestore.
private FirebaseFirestore db;

@Override
protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main);

 // getting our instance
 // from Firebase Firestore.
 db = FirebaseFirestore.getInstance();

 // initializing our edittext and buttons
 courseNameEdt = findViewById(R.id.idEdtCourseName);
 courseDescriptionEdt = findViewById(R.id.idEdtCourseDescription);
 courseDurationEdt = findViewById(R.id.idEdtCourseDuration);
 submitCourseBtn = findViewById(R.id.idBtnSubmitCourse);

 // adding on click listener for button
 submitCourseBtn.setOnClickListener(new View.OnClickListener() {
 @Override
 public void onClick(View v) {

 // getting data from edittext fields.
 courseName = courseNameEdt.getText().toString();
 courseDescription = courseDescriptionEdt.getText().toString();
 courseDuration = courseDurationEdt.getText().toString();

 // validating the text fields if empty or not.
 if (TextUtils.isEmpty(courseName)) {
 courseNameEdt.setError("Please enter Course Name");
 } else if (TextUtils.isEmpty(courseDescription)) {
 courseDescriptionEdt.setError("Please enter Course
Description");
 } else if (TextUtils.isEmpty(courseDuration)) {
 courseDurationEdt.setError("Please enter Course
Duration");
 } else {
 // calling method to add data to Firebase Firestore.
 addToFirestore(courseName, courseDescription,
courseDuration);
 }
 }
 });
}

```

```

 });
}

private void addDataToFirestore(String courseName, String
courseDescription, String courseDuration) {

 // creating a collection reference
 // for our Firebase Firestore database.
 CollectionReference dbCourses = db.collection("Courses");

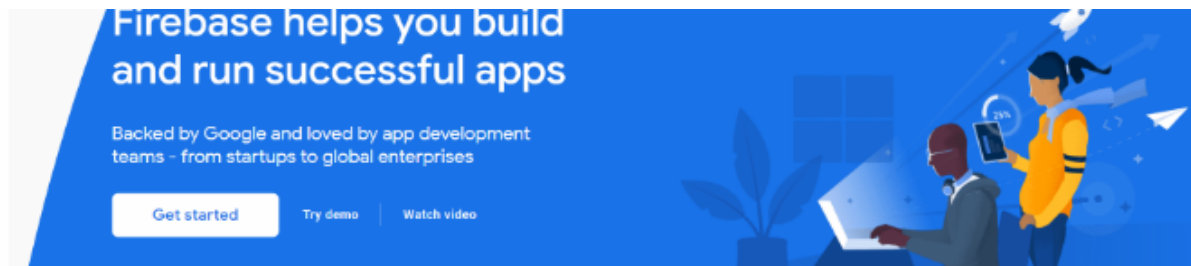
 // adding our data to our courses object class.
 Courses courses = new Courses(courseName, courseDescription,
courseDuration);

 // below method is use to add data to Firebase Firestore.
 dbCourses.add(courses).addOnSuccessListener(new
OnSuccessListener<DocumentReference>() {
 @Override
 public void onSuccess(DocumentReference documentReference) {
 // after the data addition is successful
 // we are displaying a success toast message.
 Toast.makeText(MainActivity.this, "Your Course has been added
to Firebase Firestore", Toast.LENGTH_SHORT).show();
 }
 }).addOnFailureListener(new OnFailureListener() {
 @Override
 public void onFailure(@NonNull Exception e) {
 // this method is called when the data addition process is
failed.

 // displaying a toast message when data addition is failed.
 Toast.makeText(MainActivity.this, "Fail to add course \n" + e,
Toast.LENGTH_SHORT).show();
 }
 });
}
}

```

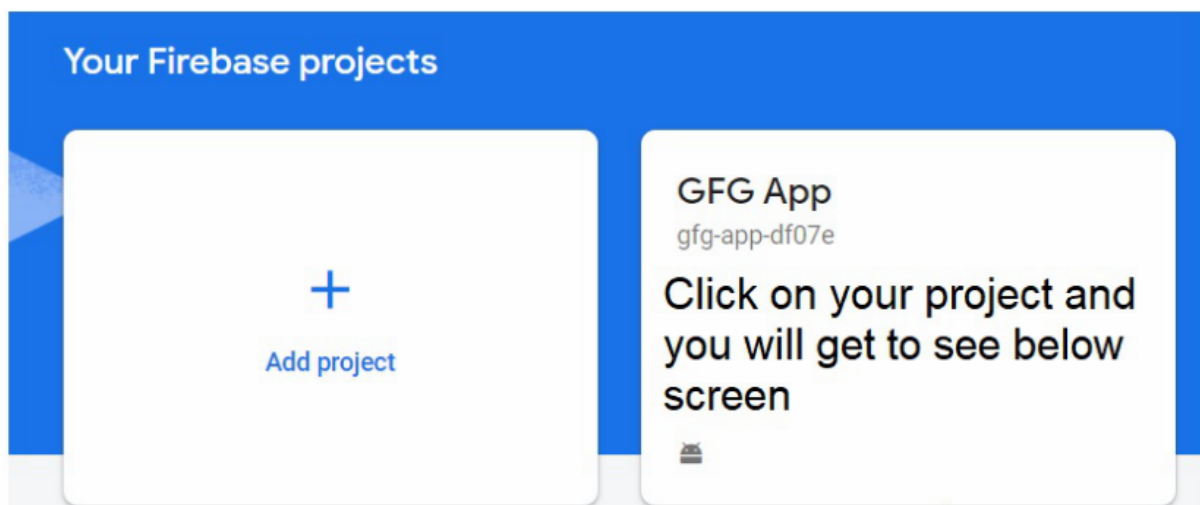
After adding this code go to this link to open [Firebase\(https://firebase.google.com/\)](https://firebase.google.com/). After clicking on this link you will get to see the below page and on this page Click on Go to Console option in the top right corner.



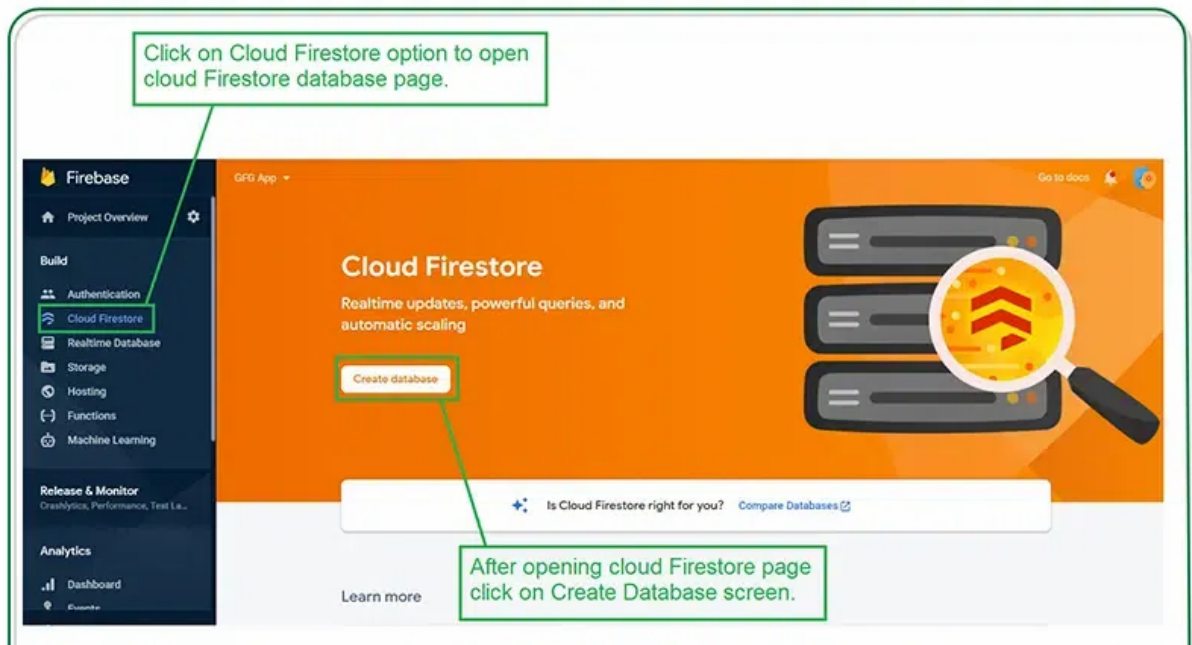
After clicking on this screen you will get to see the below screen with your all project inside that select your project.



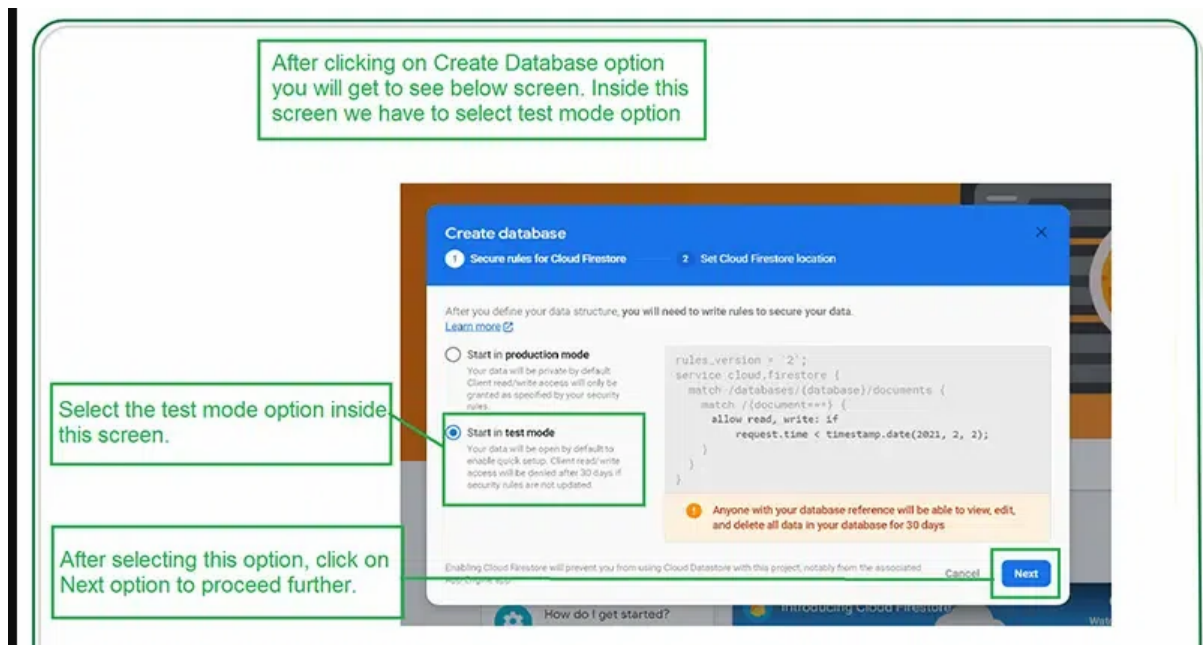
After clicking on this screen you will get to see the below screen with your all project inside that select your project.



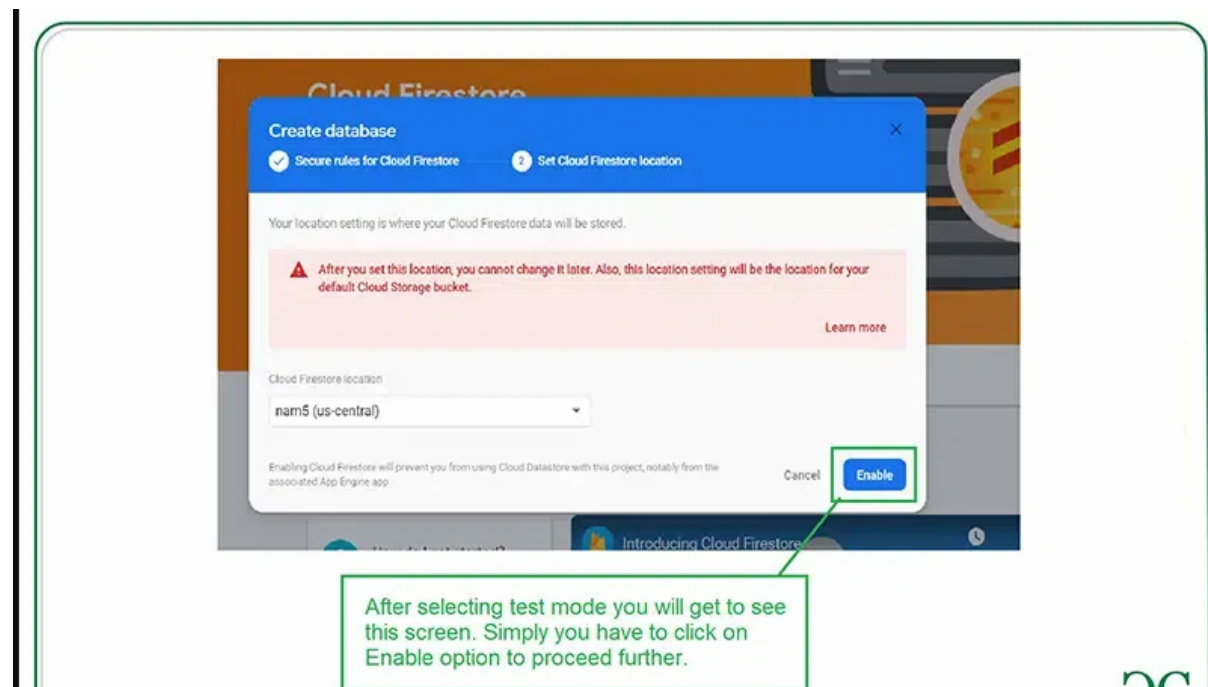
Inside that screen click n Firebase Firestore Database in the left window.



After clicking on the Create Database option you will get to see the below screen.



Inside this screen, we have to select the Start in test mode option. We are using test mode because we are not setting authentication inside our app. So we are selecting Start in test mode. After selecting on test mode click on the Next option and you will get to see the below screen.



Inside this screen, we just have to click on the Enable button to enable our Firebase Firestore database. After completing this process we just have to run our application and add data inside our app and click on the submit button. You will get to see the data added inside the Firebase Console.

Our data has been added to Firebase Firestore console.

This is our collection reference.

This is our course description inside our Firebase Console.

The screenshot shows the Firebase Firestore console interface. The breadcrumb navigation at the top reads: Home > Courses > jWubRTJekT2P... The main content area is divided into three panes. The left pane shows the project 'gfg-app-df07e' and the 'Courses' collection. The middle pane shows a document with ID 'jWubRTJekT2P4z7aId4j'. The right pane shows the document's fields: 'courseDescription' with the value 'self paced DSA Course.', 'courseDuration' with the value '30', and 'courseName' with the value 'DSA'. Green callout boxes with lines pointing to specific elements provide context: one points to the 'Courses' collection in the left pane, another to the 'courseDescription' field, a third to the 'courseDuration' field, and a fourth to the 'courseName' field.

Project	Collection	Document ID	Fields
gfg-app-df07e	Courses	jWubRTJekT2P4z7aId4j	- courseDescription: "self paced DSA Course."   - courseDuration: 30   - courseName: "DSA"

This is our course duration in Firebase Console.

This is our Course Name inside Firebase Console